

Ejercicio 1- Construcción de Imagen Docker

Multifase de Tres Etapas

Este ejercicio tiene como objetivo consolidar las mejores prácticas de seguridad y optimización vistas en el módulo, específicamente la técnica de **Imágenes Multifase** (*Multistage Builds*). El reto consiste en crear una imagen de producción extremadamente pequeña y segura, utilizando una aplicación escrita en Go, un lenguaje compilado.

I. Objetivo del Ejercicio

El ejercicio requiere la creación de un **Dockerfile multifase** con **tres etapas** distintas, según el diagrama:



1. **Stage 1: Compilation:** Compilar la aplicación Go para obtener un binario estático.
2. **Stage 2: Compression:** Reducir aún más el tamaño del binario utilizando una herramienta de compresión (UPX).
3. **Stage 3: Final Image Distroless:** Utilizar la imagen mínima y segura `distroless/static` y copiar solo el binario comprimido final.

El propósito principal es demostrar que las imágenes multifase pueden tener más de dos etapas y fomentar la familiarización con esta práctica recomendada para la optimización y la seguridad.

II. Aplicación Base y Requisitos

La aplicación utilizada es un servidor web básico escrito en Go, cuyo código fuente se encuentra en el repositorio.

- **Aplicación (`main.go`):** Un servidor web simple que responde "Hello world" en el *endpoint* raíz (/). Escucha en el puerto `8080`.

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

func handler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello world")
}

func main() {
    http.HandleFunc("/", handler)
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

- **Go Module (`go.mod`):** Define el módulo y la versión de Go a utilizar (`go 1.22.0`).

```
module multistage
```

```
go 1.22.0
```

III. Resolución del Ejercicio: Dockerfile Multifase de Tres Etapas

El proceso de construcción se divide para aislar las dependencias grandes (compilador) y transitorias (herramienta de compresión) de la imagen final de ejecución.

Paso 1: Clonación del Código Fuente

Antes de construir la imagen, se descarga la aplicación Go del repositorio.

Bash

```
git clone https://github.com/cyberhades/singhack
```

```
cd singhack
```

Paso 2: Creación del Dockerfile con Tres Etapas

```
FROM golang:1.22 as build
WORKDIR /go/src/app
COPY . .
RUN CGO_ENABLED=0 go build -o /app

FROM ubuntu as compress
RUN apt update && apt install upx -y
COPY --from=build /app /app
RUN upx -o /appz /app

FROM gcr.io/distroless/static
COPY --from=compress /appz /app
EXPOSE 8080
CMD ["/app"]
```

Estamos creando un **Dockerfile de múltiples etapas** (**multi-stage Dockerfile**) diseñado para crear un binario de Go, optimizar su tamaño y

ejecutarlo en el contenedor más pequeño y seguro posible. El proceso se divide en tres etapas: **construcción**, **compresión** y **ejecución final (distroless)**.

1. Etapa de Construcción (**build**)

Esta etapa se enfoca en compilar el código fuente de Go para producir un binario ejecutable estático:

- La primera instrucción es **FROM golang:1.22 as build**. Esto establece la imagen oficial de Go versión 1.22 como base para esta etapa y le asigna el alias **build**.
 - A continuación, **WORKDIR /go/src/app** establece el directorio de trabajo dentro del contenedor.
 - **COPY . .** copia todo el código fuente del contexto de construcción (tu directorio local) al directorio de trabajo del contenedor.
 - Finalmente, **RUN CGO_ENABLED=0 go build -o /app** compila la aplicación. La variable de entorno **CGO_ENABLED=0** es crucial, ya que asegura que el binario resultante se enlace estáticamente, haciéndolo independiente de las librerías del sistema operativo (como **glibc**), lo cual es necesario para la etapa final *distroless*. El binario compilado se guarda en **/app**.
-

2. Etapa de Compresión (**compress**)

Esta etapa se utiliza para reducir aún más el tamaño del binario antes de la ejecución:

- Se inicia una nueva etapa con **FROM ubuntu as compress**, usando la imagen base de Ubuntu y asignándole el alias **compress**. Esto se hace porque la imagen de Go no incluye la herramienta de compresión que se necesita.

- **RUN apt update && apt install upx -y** instala la herramienta de compresión de ejecutables **upx**.
 - **COPY --from=build /app /app** copia el binario compilado (**/app**) desde la etapa anterior (**build**) a esta nueva etapa.
 - **RUN upx -o /appz /app** ejecuta **upx** para comprimir el binario **/app** y guarda la versión comprimida como **/appz**.
-

3. Etapa Final y de Ejecución (**distroless**)

Esta es la etapa final que resulta en la imagen de producción, priorizando la seguridad y el tamaño:

- Se inicia la etapa final con **FROM gcr.io/distroless/static**.
distroless es una imagen mínima de Google que no contiene *shell*, gestor de paquetes (**apt**) ni archivos innecesarios, lo cual minimiza drásticamente la superficie de ataque. Solo puede ejecutar binarios estáticamente enlazados.
- **COPY --from=compress /appz /app** copia el binario **comprimido** (**/appz**) de la etapa **compress** a la ruta **/app** de la imagen final.
- **EXPOSE 8080** documenta que el contenedor está escuchando peticiones en el puerto 8080.
- **CMD ["/app"]** establece el punto de entrada predeterminado del contenedor para que ejecute el binario de la aplicación.

El propósito general de estas tres etapas es asegurar que la imagen de Docker final (**distroless**) solo contenga el binario comprimido de la aplicación, resultando en una imagen de **producción mínima, segura y altamente optimizada en tamaño**.

Paso 3: Construcción y Prueba Final

Construir la Imagen:

```
docker build -t exercise1 .
```

- Durante el proceso, Docker utiliza la caché de las etapas *build* y *compress* si no hay cambios, y luego crea la capa final *distroless*.

Ejecutar y Mapear Puertos:

```
docker run -it -p 8080:8080 exercise1
```

Verificación Funcional: Se comprueba que la aplicación comprimida y ejecutada en el entorno mínimo *distroless* sigue funcionando correctamente.

```
curl localhost:8080
```

```
# Resultado esperado: Hello world
```

IV. Conclusión (Observación de la Compresión)

Para observar el efecto de la compresión, se puede comparar el tamaño del binario original (*/app*) con el comprimido (*/appz*) en la Etapa 2. Aunque la compresión no es una práctica universal en producción, este ejemplo demuestra:

1. **Flexibilidad de Multifase:** La capacidad de utilizar **múltiples fases** (más de dos) para tareas intermedias (compilación, pruebas, compresión).
2. **Optimización Estricta:** La técnica de copiar solo el artefacto final asegura que la imagen de producción sea lo más pequeña y segura posible.

Ejercicio 2 - Seguridad Profunda: Ejecución de Contenedores Non-Root en Imágenes Distroless (Ejercicio 2)

El segundo ejercicio avanzado de Docker consolida las mejores prácticas de **seguridad** y **optimización** al requerir la ejecución de un binario en una imagen **Distroless** (segura y mínima) bajo un usuario con **privilegios mínimos** (*non-root*). Esto exige una manipulación avanzada del flujo Multifase, ya que las imágenes Distroless carecen de las herramientas necesarias para la gestión de usuarios.

I. Propuesta del Ejercicio y Desafío de Seguridad

El ejercicio se basa en la imagen multifase de tres etapas construida previamente (Compilación, Compresión y Destino Distroless), añadiendo un requisito crucial:



- **Requisito:** La **Fase 3 (Final image Distroless)** debe ejecutar el binario comprimido **Running as non root** (con un usuario que no sea *root*).
- **Desafío:** Las imágenes gcr.io/distroless/static no incluyen la *shell* (`/bin/sh`) ni herramientas de sistema como `useradd`. Intentar ejecutar `RUN useradd` en la fase Distroless causará un fallo en la construcción.

- **Solución Implícita:** La creación del usuario y la copia de su metadata de identificación deben realizarse en una **fase anterior** (como la fase de `build` o `compress`) y luego copiarse a la fase final.

II. Resolución del Ejercicio: Manipulación de Fases y Archivos de Usuario

La solución requiere modificar la fase de `build` para crear el usuario y luego usar comandos `COPY --from` para transferir los archivos de identificación del usuario (`/etc/passwd` y `/etc/group`) a la imagen final.

Paso 1: Fallo en la Etapa Distroless (Demostración)

Intentar crear el usuario en la fase Distroless falla debido a la ausencia de la `shell` y el binario `useradd`.

Intento Inviabile:

```
FROM golang:1.22 as build
```

```
WORKDIR /go/src/app
```

```
COPY . .
```

```
RUN CGO_ENABLED=0 go build -o /app
```

```
FROM ubuntu as compress
```

```
RUN apt update && apt install upx -y
```

```
COPY --from=build /app /app
```

```
RUN upx -o /appz /app
```

```
FROM gcr.io/distroless/static
```

```
RUN useradd --no-log-init --system --shell /usr/sbin/nologin
```

```
appuser # modificación
```

```
COPY --from=compress /appz /app
```

```
EXPOSE 8080
```

```
USER appser # modificación
```

```
CMD ["/app"]
```


El error principal es que se intenta ejecutar el comando **useradd** dentro de la imagen **distroless/static**, la cual no contiene la *shell* necesaria (**/bin/sh**) ni el propio binario **useradd**.

La **línea importante** y donde se produce el error es la **Línea del Dockerfile** que contiene la instrucción **RUN useradd**

El mensaje de error clave, que aparece tanto en la salida de la terminal como al final del *build*, es:

```
0.552 runc runc failed: unable to start container process:
exec: "/bin/sh": stat /bin/sh: no such file or directory
```

Explicación del Error

1. **Instrucción RUN:** Cuando Docker ejecuta una instrucción **RUN** en el formato *shell* (sin corchetes **[" "]**), intenta envolver el comando con la *shell* predeterminada, que es **/bin/sh** (es decir, internamente se ejecuta **/bin/sh -c useradd ...**).
2. **Imagen distroless/static:** La imagen base utilizada en esa etapa es **gcr.io/distroless/static**. Por diseño, las imágenes *distroless* **no incluyen shells** de propósito general (como **/bin/sh**) ni la mayoría de las herramientas estándar del sistema operativo (como **useradd** o **apt**) por razones de seguridad y tamaño.
3. **Fallo:** Docker no encuentra **/bin/sh** para ejecutar el comando **useradd**, y por lo tanto, la construcción falla.

Solución Implícita: Si fuera absolutamente necesario añadir un usuario, no se podría hacer en la etapa **distroless**. La creación de usuarios debe hacerse en una etapa anterior con una imagen que tenga una *shell* y el binario **useradd** (como **ubuntu** o la imagen de Go), y luego solo copiar los resultados (si aplica) a la etapa *distroless*. No obstante, el propósito de *distroless* es a menudo ejecutar un binario

estático como `root` o usar la funcionalidad nativa de Kubernetes para gestionar el usuario.

Paso 2: Creación del Usuario en una Fase Previa (Fase 1: Build)

La creación del usuario se mueve a la **Etapas 1 (build)**, donde existe un sistema operativo funcional (GoLang, basado en Debian/Ubuntu).

Creación del Usuario Non-Root: Se añade la instrucción `RUN useradd` en la Fase 1. El uso de `--system` y `--shell /usr/sbin/nologin` minimiza los privilegios del nuevo usuario.

```
# Fase 1
```

```
FROM golang:1.22 as build
```

```
WORKDIR /go/src/app
```

```
COPY . .
```

```
RUN CGO_ENABLED=0 go build -o /app
```

```
RUN useradd --no-log-init --system --shell /usr/sbin/nologin  
appuser # modificación
```

Paso 3: Copia de Metadatos de Usuario a la Imagen Distroless

Aunque el binario `useradd` crea el usuario, la imagen final `distroless` necesita los archivos de configuración (`/etc/passwd` y `/etc/group`) que contienen el registro del nuevo usuario (`appuser`) para poder validar su ID.

1. **Copia desde una Fase No Inmediata:** Se utiliza `COPY --from=build` para copiar los archivos directamente desde la Fase 1 a la Fase 3, demostrando que se puede interactuar con **cualquier fase previa**, no solo con la inmediatamente anterior.
 - **Archivos a Copiar:**

- `/etc/passwd` (Contiene la lista de usuarios).
- `/etc/group` (Contiene la lista de grupos).

Fase 3

```
FROM gcr.io/distroless/static
COPY --from=compress /appz /app
COPY --from=build /etc/passwd /etc/passwd
COPY --from=build /etc/group /etc/group
EXPOSE 8080
USER appser

CMD ["/app"]
```

Establecer el Usuario de Ejecución: Se establece el usuario *non-root* con el que se ejecutará la aplicación.

```
USER appser
```

Contenido Final del Dockerfile (Exercise 2)

El Dockerfile final integra la optimización del binario con la seguridad de usuario:

```
FROM golang:1.22 as build
WORKDIR /go/src/app
COPY . .
RUN CGO_ENABLED=0 go build -o /app
RUN useradd --no-log-init --system --shell /usr/sbin/nologin
appuser # modificación
```

```
FROM ubuntu as compress
RUN apt update && apt install upx -y
COPY --from=build /app /app
RUN upx -o /appz /app
```

```
FROM gcr.io/distroless/static
```

```
COPY --from=compress /appz /app
COPY --from=build /etc/passwd /etc/passwd
COPY --from=build /etc/group /etc/group
EXPOSE 8080
USER appser

CMD ["/app"]
```

III. Construcción y Verificación

Construir la Imagen:

```
docker build -t exercise2 .
```

Ejecutar el Contenedor y Comprobar Funcionalidad:

```
docker run -it -p 8080:8080 exercise2
```

```
curl localhost:8080 # La aplicación sigue funcionando.
```

Verificación del Usuario Non-Root: Para confirmar que la aplicación se ejecuta con el usuario de bajos privilegios (ya que la imagen es `distroless` y no tiene `shell` para usar `docker exec bash`), se debe inspeccionar la imagen final.

```
docker inspect exercise2
```

1. *Resultado:* La sección `Config` del `inspect` confirmará que el valor de `User` ha sido establecido a `appuser` (o el ID de usuario correspondiente).

IV. Conclusiones

- **Multifase y Seguridad:** El ejercicio demuestra que la implementación de la buena práctica de ejecutar procesos como **non-root** en imágenes

minimalistas (`distroless`) requiere el uso avanzado de **múltiples fases** y comandos `COPY --from`.

- **Superación de Limitaciones:** Se solventa la limitación de la imagen `distroless` (que carece de `useradd` y `shell`) ejecutando la lógica de creación de usuario en una fase previa (`build`) y copiando solo la **metadata de identificación** (`/etc/passwd`, `/etc/group`) a la imagen final.
- **Buena Práctica:** Es una muy buena práctica de seguridad ejecutar contenedores con usuarios que no sean el *root*.
- **Interacción Multifase:** Se confirma que es posible interactuar y copiar ficheros de **cualquier fase previa** definida en el Dockerfile, y no solo de la fase inmediatamente anterior.

Ejercicio 3 - Orquestación Segura y Distribución:

Despliegue Completo de Aplicaciones en Kubernetes

El tercer ejercicio integra todo el conocimiento adquirido en el módulo, abarcando el ciclo de vida completo de una aplicación segura: desde la **construcción optimizada** de la imagen, pasando por su **distribución** a un registro, hasta su **despliegue en Kubernetes** y la **exposición externa** mediante **Ingress** con un nombre de dominio específico.

I. Requisitos del Ejercicio 3

La tarea consiste en tomar la imagen segura y optimizada del ejercicio anterior y hacerla accesible a usuarios externos.

1. **Subir la Imagen:** Publicar la imagen `exercise2` en un registro de Docker (ej. Docker Hub).
2. **Despliegue en Kubernetes:** Crear un Pod que consuma la imagen del registro.
3. **Exposición Externa:** Exponer la aplicación de forma que los usuarios puedan acceder a ella a través de la URL `http://exercise3.lab.local`.

II. Procedimientos Prácticos de Distribución

El primer paso es preparar y publicar la imagen final (`exercise2`) en el registro para que Kubernetes pueda descargarla.

Reconstruir y Etiquetar la Imagen (Nomenclatura Completa): Para subir la imagen al registro, debe tener el prefijo de la cuenta/repositorio del usuario. Se recomienda reconstruir o re-etiquetar la imagen con la nomenclatura completa.

Reconstruir directamente con el tag de la cuenta

```
docker build -t cyberhades/exercise2 .
```

Enviar la Imagen al Registro (`docker push`): Se utiliza el comando `docker push` para subir la imagen al repositorio especificado.

```
docker push cyberhades/exercise2
```

- *Propósito:* Al estar la imagen en el registro, el clúster de Kubernetes puede descargarla en cualquier momento y en cualquier Nodo.

III. Despliegue en Kubernetes: Pod y Service

Una vez que la imagen está disponible, se define la infraestructura en Kubernetes.

1. Despliegue del Pod (Unidad de Ejecución)

Se define un Pod simple que ejecuta la imagen recién subida.

Fichero `pod.yaml`:

```
apiVersion: v1

kind: Pod
metadata:
  name: exercise3
  labels:
    app: exercise3
spec:
  containers:
    - name: exercise3
      image: cyberhades/exercise2 # imagen del registro
      ports:
        - containerPort: 8080 # puerto interno
```

Este manifiesto define un **Pod**, que es la unidad más pequeña y fundamental en Kubernetes, conteniendo uno o más contenedores.

Metadatos del Objeto

- **apiVersion: v1** y **kind: Pod**: Estas líneas identifican que estamos creando el objeto base de Kubernetes, un **Pod**, usando la versión principal de la API.
- **metadata:**
 - **name: exercise3**: Asigna el nombre **exercise3** a este Pod.
 - **labels: app: exercise3**: Aplica una **etiqueta** al Pod. Estas etiquetas son cruciales, ya que permiten que otros recursos (como *Services* o *Deployments*) identifiquen y se dirijan a este Pod.

Especificación del Contenedor (**spec**)

La sección **spec** describe el contenido del Pod, en este caso, un único contenedor.

- **containers:**: Una lista de contenedores que se ejecutarán dentro del Pod.
 - **- name: exercise3**: Asigna el nombre **exercise3** al contenedor (diferente al nombre del Pod, aunque a menudo coinciden).
 - **image: cyberhades/exercise2**: Especifica la imagen de Docker que se debe usar para el contenedor. Kubernetes la descargará de un registro (como Docker Hub).
 - **ports: - containerPort: 8080**: Informa a Kubernetes que el contenedor espera recibir tráfico en el **puerto 8080**. Es importante destacar que definir **containerPort** solo expone el puerto *internamente* dentro de la red del Pod; para acceder al Pod desde fuera del clúster o desde otros Pods, se necesita un recurso **Service** que lo apunte.

Aplicar el Pod:

```
kubectl apply -f pod.yaml
```



```
kubectl get po # Comprobar que esté Running
```

2. Creación del Servicio ClusterIP (Exposición Interna)

Para exponer el Pod de forma estable dentro del clúster, se crea un Servicio del tipo **ClusterIP**.

Fichero **service.yaml**:

```
apiVersion: v1

kind: Service
metadata:
  name: exercise3
spec:
  selector:
    app: exercise3 # Debe coincidir con el label del pod
  ports:
    - port: 8080 # Puerto del servicio (TargetPort: 8080 implícito)
```

Este manifiesto define un **Servicio** (**kind: Service**) que actúa como una dirección de red estable y un balanceador de carga para un conjunto de *Pods*.

- **apiVersion: v1** y **kind: Service**: Identifican el tipo de objeto que se está creando.
- **metadata: name: exercise3**: Asigna el nombre **exercise3** al Servicio. Este será el nombre DNS interno que otros *Pods* usarán para comunicarse con esta aplicación.
- **spec: selector: app: exercise3**: Esta es la regla de **selección** principal. El Servicio dirigirá el tráfico a **cualquier Pod que tenga la etiqueta app: exercise3**.
- **ports: - port: 8080**: Configura el puerto del Servicio. Indica que el Servicio escucha en el **puerto 8080**. Por defecto, se asume que los *Pods*

seleccionados también están escuchando en el puerto **8080** (el `targetPort` implícito).

Aplicar el Servicio:

```
kubectl apply -f service.yaml
```

Verificación de Endpoints: Se comprueba que el Servicio (`svc`) haya encontrado un *endpoint* (la IP del Pod).

```
kubectl describe svc exercise3
```

- *Propósito:* Se confirma que el tráfico dirigido al Servicio en el puerto 8080 será balanceado a la IP interna del Pod.

IV. Exposición Externa mediante Ingress

Para cumplir con el requisito de acceso mediante un nombre de dominio (`exercise3.lab.local`), se crea un recurso **Ingress** que utiliza el Servicio ClusterIP como *backend*.

- **Requisito:** El *addon* de Ingress de Minikube debe estar habilitado.

Fichero `ingress.yaml`:

```
apiVersion: networking.k8s.io/v1

kind: Ingress
metadata:
  name: exercise3
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$1
spec:
  rules:
    - host: exercise3.lab.local
      http:
```

```
paths:
  - path: /
    pathType: Prefix
    backend:
      service:
        name: exercise3 # APunta al Service Cluster IP
        port:
          number: 8080
```

Este manifiesto configura un recurso **Ingress** para exponer la aplicación **exercise3** a través de una URL específica fuera del clúster .

1. **apiVersion y kind**: Identifican el objeto como un **Ingress** que utiliza la API de redes.
2. **metadata: name: exercise3**: Nombra el recurso **Ingress**.
3. **annotations: nginx.ingress.kubernetes.io/rewrite-target: /\$1**: Esta es una anotación específica para el controlador NGINX Ingress. Indica que, después de enrutar el tráfico, el **path original de la URL debe ser reescrito**, dirigiendo siempre la petición a la raíz (/) del servicio *backend* (esta es una técnica común para simplificar el *routing*).
4. **spec: rules::** Define las condiciones de enrutamiento.
5. **host: exercise3.lab.local**: El *Ingress* solo procesará las peticiones que lleguen al nombre de host **exercise3.lab.local**.
6. **paths: - path: /**: Dentro de ese host, cualquier petición que comience con la ruta / (**pathType: Prefix**) se enrutará.
7. **backend::** Define el destino del tráfico.
 - **service: name: exercise3**: El tráfico se envía al **Service** interno llamado **exercise3**.
 - **port: number: 8080**: El tráfico se dirige al puerto **8080** de ese servicio.

Aplicar el Ingress:

```
kubectl apply -f ingress.yaml
```

- Se utiliza `kubectl get ing -w` para esperar la asignación de la IP (la IP del Nodo Minikube).

V. Métodos de Acceso (Debugging y Testeo)

Dado que el dominio `exercise3.lab.local` no existe públicamente, se necesitan trucos para simular la resolución DNS en el entorno de desarrollo.

Acceso mediante Cabecera `Host` (Método CLI): Se realiza una petición directamente a la IP de Minikube, forzando la cabecera `Host` a `exercise3.lab.local` para que el Ingress sepa qué regla aplicar.

```
curl -H "Host: exercise3.lab.local" ${minikube ip}
```

Acceso mediante Modificación de `hosts` (Método Sencillo): El método más limpio para el testeo es modificar el archivo de `hosts` del sistema anfitrión para que resuelva el dominio localmente.

Acción: Añadir una nueva entrada al fichero `/etc/hosts` con permisos de `root`.
`sudo vi /etc/hosts`

Añadir la IP del minikube y el dominio

```
192.168.49.2 exercise3.lab.local
```

Verificación: Una vez modificado, el acceso puede realizarse directamente usando el dominio en la CLI o navegador.

```
curl exercise3.lab.local
```

VI. Conclusiones

El ejercicio demuestra el flujo de trabajo completo y las buenas prácticas profesionales:

- **Distribución:** La imagen se etiqueta correctamente y se sube al registro (`docker push`) como paso previo al despliegue.
- **Estabilidad y Exposición:** Se utiliza un **Service ClusterIP** para proporcionar una **dirección de red estable** interna, y luego el **Ingress** para exponer esa estabilidad de forma externa, utilizando reglas de Capa 7 (dominio).
- **Entornos de Desarrollo:** Se utilizan herramientas locales (modificación de `/etc/hosts`) para simular la resolución DNS que se requeriría en un entorno de producción.